

Установка пакетов

↑ Наверх

В этом разделе рассказывается об основах установки Python [пакетов](#).

Важно отметить, что термин «пакет» в данном контексте используется для описания набора программного обеспечения, которое необходимо установить (то есть как синоним [дистрибутива](#)). Он не относится к [пакетам](#), которые вы импортируете в свой исходный код на Python (то есть к контейнерам с модулями). В сообществе Python принято называть [дистрибутив](#) «пакетом». Термин «дистрибутив» часто не используют, потому что его легко спутать с дистрибутивом Linux или другим крупным дистрибутивом программного обеспечения, например с самим Python.

Требования к установке пакетов

В этом разделе описаны действия, которые необходимо выполнить перед установкой других пакетов Python.

Убедитесь, что вы можете запустить Python из командной строки

Прежде чем продолжить, убедитесь, что у вас установлен Python и что в вашей командной строке доступна нужная версия. Это можно проверить, выполнив следующую команду:

[Unix/macOS](#) [Windows](#)

```
python3 --version
```

Вы должны получить примерно такой результат: `Python 3.6.3`. Если у вас нет Python, пожалуйста, установите последнюю версию 3.x с [python.org](#) или обратитесь к разделу [Установка Python](#) в «Автостопе по Python».

Примечание

Если вы новичок и столкнулись с подобной ошибкой:

```
Traceback (most recent call last):version --python3
>>>
Файл "<stdin>", строка 1, в <модуле>
NameError: имя 'python3' не определено
```

Это связано с тем, что данная команда и другие команды, предложенные в этом руководстве,

[Перейти к контенту](#)

лнения в оболочке (также называемой *терминалом*) вашей операционной системы и взаимодейст

[руководстве](#) по началу работы с Python для начинающих.

🌐 en



Последние



Примечание

Если вы используете расширенную оболочку, например IPython или Jupyter Notebook, вы можете запускать системные команды, подобные тем, что описаны в руководстве, добавляя к ним символ `!` в начале:

↑ Наверх

```
In [1]: import sys
!{sys.executable} --version
Python 3.6.3
```

Рекомендуется писать `{sys.executable}` вместо обычного `python`, чтобы команды выполнялись в той версии Python, которая соответствует запущенному блокноту (это может быть не та версия Python, на которую указывает команда `python`).

Примечание

Из-за того, что большинство дистрибутивов Linux справляются с переносом Python 3, пользователям Linux, использующим системный Python без предварительного создания виртуальной среды, следует заменить `python` команду в этом руководстве на `python3`, а `python -m pip` команду на `python3 -m pip --user`. Не выполняйте ни одну из команд из этого руководства с помощью `sudo`: если вы получаете ошибку разрешений, вернитесь к разделу о создании виртуальных сред, настройте ее, а затем продолжайте выполнять руководство, как написано.

Убедитесь, что вы можете запустить `pip` из командной строки

Кроме того, вам нужно убедиться, что у вас установлен [pip](#). Это можно проверить, выполнив следующую команду:

[Unix/macOS](#) [Windows](#)

```
python3 -m pip --version
```

Если вы установили Python из исходного кода, с помощью установщика с [python.org](#) или через [Homebrew](#), у вас уже должен быть `pip`. Если вы используете Linux и устанавливаете Python с помощью менеджера пакетов вашей операционной системы, возможно, вам придется установить `pip` отдельно. См. [Установка pip/setuptools/wheel с помощью менеджеров пакетов Linux](#).

Если `pip` еще не установлен, сначала попробуйте загрузить его из стандартной библиотеки:

[Unix/macOS](#) [Windows](#)

```
python3 -m ensurepip --default-pip
```

[Перейти к контенту](#)

 [en](#)  [Последние](#) ▼

Если это по-прежнему не позволяет вам запустить `python -m pip`:

- Безопасно скачайте [get-pip.py](#) ^[1]
- Запустите `python get-pip.py`. ^[2] Это у [↑ Наверх](#) ли обновит `pip`. Кроме того, будут установлены [Setuptools](#) и [wheel](#), если они еще не установлены.

Предупреждение

Будьте осторожны, если используете установку Python, управляемую вашей операционной системой или другим менеджером пакетов. `get-pip.py` не взаимодействует с этими инструментами и может привести к нестабильной работе системы. Вы можете использовать `python get-pip.py --prefix=/usr/local/` для установки в `/usr/local` — этот способ предназначен для локально установленного программного обеспечения.

Убедитесь, что `pip`, `setuptools` и `wheel` обновлены до последней версии

Хотя для установки из готовых бинарных архивов достаточно одного `pip`, актуальные копии проектов `setuptools` и `wheel` пригодятся, чтобы вы могли установить их из исходных архивов:

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install --upgrade pip setuptools wheel
```

При желании создайте виртуальную среду

Подробнее см. [в разделе ниже](#), но вот базовая команда `venv` ^[3], которую можно использовать в обычной системе Linux:

[Unix/macOS](#) [Windows](#)

```
python3 -m venv tutorial_env
source tutorial_env/bin/activate
```

Это создаст новую виртуальную среду в подкаталоге `tutorial_env` и настроит текущую оболочку на использование этой среды `python` по умолчанию.

Создание виртуальных сред

«Виртуальные среды» Python позволяют устанавливать [пакеты](#) Python в изолированном месте для конкретного приложения, а не глобально. Если вам нужно безопасно установить глобальные инструменты командной строки, ознакомьтесь с [инструкцией по установке и использованию их инструментов командной строки](#).

[Перейти к контенту](#)

[en](#) [Последние](#) ▼

Представьте, что у вас есть приложение, которому нужна версия 1 библиотеки LibFoo, а другому приложению требуется версия 2. Как вам использовать оба этих приложения? Если вы устанавливаете все в `/usr/lib/python2.7/site-packages` (или в другое стандартное расположение на вашей платформе), то вы можете столкнуться с ситуацией, когда вы непреднамеренно обновите приложение, которое не нужно обновлять.

Or more generally, what if you want to install an application and leave it be? If an application works, any change in its libraries or the versions of those libraries can break the application.

Also, what if you can't install [packages](#) into the global site-packages directory? For instance, on a shared host.

In all these cases, virtual environments can help you. They have their own installation directories and they don't share libraries with other virtual environments.

Currently, there are two common tools for creating Python virtual environments:

- [venv](#) is available by default in Python 3.3 and later, and installs [pip](#) into created virtual environments in Python 3.4 and later (Python versions prior to 3.12 also installed [Setuptools](#)).
- [virtualenv](#) needs to be installed separately, but supports Python 2.7+ and Python 3.3+, and [pip](#), [Setuptools](#) and [wheel](#) are installed into created virtual environments by default. Note that `setuptools` is no longer included by default starting with Python 3.12 (and `virtualenv` follows this behavior).

The basic usage is like so:

Using [venv](#):

[Unix/macOS](#) [Windows](#)

```
python3 -m venv <DIR>
source <DIR>/bin/activate
```

Using [virtualenv](#):

[Unix/macOS](#) [Windows](#)

```
python3 -m virtualenv <DIR>
source <DIR>/bin/activate
```

For more information, see the [venv](#) docs or the [virtualenv](#) docs.

The use of **source** under Unix shells ensures that the virtual environment's variables are set within the current shell, and not in a subprocess (which then disappears, having no useful effect).

[Перейти к контенту](#) es, Windows users should *not* use the :
script directly from the command shell I

en



Последние



```
<DIR>\Scripts\activate
```

Managing multiple virtual environments directly becomes tedious, so the [dependency management tutorial](#) introduces a higher level tool, [Pipenv](#), that automatically manages a separate virtual environment for each project and application that you work on.

Use pip for Installing

[pip](#) is the recommended installer. Below, we'll cover the most common usage scenarios. For more detail, see the [pip docs](#), which includes a complete [Reference Guide](#).

Installing from PyPI

The most common usage of [pip](#) is to install from the [Python Package Index](#) using a [requirement specifier](#). Generally speaking, a requirement specifier is composed of a project name followed by an optional [version specifier](#). A full description of the supported specifiers can be found in the [Version specifier specification](#). Below are some examples.

To install the latest version of "SomeProject":

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install "SomeProject"
```

To install a specific version:

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install "SomeProject==1.4"
```

To install greater than or equal to one version and less than another:

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install "SomeProject>=1,<2"
```

To install a version that's [compatible](#) with a certain version: ^[4]

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install "SomeProject~1.4.2"
```

[Перейти к контенту](#) to install any version "~=1.4.*" version ·  en  Последние ▼

Source Distributions vs Wheels

`pip` can install from either [Source Distributions](#) or [Wheels](#), but if both are present on PyPI, `pip` will prefer a compatible [wheel](#). You can change `pip`'s default behavior by e.g. using its `--no-binary` option.

[Wheels](#) are a pre-built [distribution](#) format that provides faster installation compared to [Source Distributions \(sdist\)](#), especially when a project contains compiled extensions.

If `pip` does not find a wheel to install, it will locally build a wheel and cache it for future installs, instead of rebuilding the source distribution in the future.

Upgrading packages

Upgrade an already installed `SomeProject` to the latest from PyPI.

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install --upgrade SomeProject
```

Installing to the User Site

To install [packages](#) that are isolated to the current user, use the `--user` flag:

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install --user SomeProject
```

For more information see the [User Installs](#) section from the `pip` docs.

Note that the `--user` flag has no effect when inside a virtual environment - all installation commands will affect the virtual environment.

If `SomeProject` defines any command-line scripts or console entry points, `--user` will cause them to be installed inside the `user base`'s binary directory, which may or may not already be present in your shell's `PATH`. (Starting in version 1.0, pip displays a warning when installing any scripts to a directory outside `PATH`.) If the scripts are not available in your shell after installation, you'll need to add the directory to your `PATH`:

- On Linux and macOS you can find the user base binary directory by running `python -m site --user-base` and adding `bin` to the end. For example, this will typically print `~/.local` (with `~` expanded to the absolute path to your home directory) so you'll need to add `~/.local/bin` to your `PATH`. You can set your `PATH` permanently by [modifying ~/.profile](#).
- On Windows you can find the user base binary directory by running `py -m site --user-site` and replacing `site-packages` with `Scripts`. For example, this could return `C:\Users\Username\AppData\Roaming\Python36\site-packages` so you would need to set your `PATH` to include `C:\Users\Username\AppData\Roaming\Python36\Scripts`. You can set your user `PATH` permanently in the [Control Panel](#). You may need to log out for the `PATH` changes to take effect.

Requirements files

Install a list of requirements specified in a [Requirements File](#).

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install -r requirements.txt
```

Installing from VCS

Install a project from VCS in “editable” mode. For a full breakdown of the syntax, see pip's section on [VCS Support](#).

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install -e SomeProject @ git+https://git.repo/some_pkg.git # f
python3 -m pip install -e SomeProject @ hg+https://hg.repo/some_pkg # f
python3 -m pip install -e SomeProject @ svn+svn://svn.repo/some_pkg/trunk/ # f
python3 -m pip install -e SomeProject @ git+https://git.repo/some_pkg.git@feature # f
```

Installing from other Indexes

Install from an alternate index

[Перейти к контенту](#)

[en](#) [Последние](#) ▼

```
python3 -m pip install --index-url http://my.package.repo/simple/ SomeProject
```

Search an additional index during install, in [↑ Наверх](#) [PyPI](#)

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install --extra-index-url http://my.package.repo/simple SomeProject
```

Installing from a local src tree

Installing from local src in [Development Mode](#), i.e. in such a way that the project appears to be installed, but yet is still editable from the src tree.

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install -e <path>
```

You can also install normally from src

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install <path>
```

Installing from local archives

Install a particular source archive file.

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install ./downloads/SomeProject-1.0.4.tar.gz
```

Install from a local directory containing archives (and don't check [PyPI](#))

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install --no-index --find-links=file:///local/dir/ SomeProject
python3 -m pip install --no-index --find-links=/local/dir/ SomeProject
python3 -m pip install --no-index --find-links=relative/dir/ SomeProject
```

Installing from other sources

[Перейти к контенту](#) [ta sources](#) (for example Amazon S3 stc [en](#) [Последние](#) [▼](#))
application that presents the data in a format compliant with the [simple repository API](#)., and use the `--extra-index-url` flag to direct pip to use that index.


```
./s3helper --port=7777  
python -m pip install --extra-index-url http://localhost:7777 SomeProject
```

↑ Наверх

Installing Prereleases

Find pre-release and development versions, in addition to stable versions. By default, pip only finds stable versions.

Unix/macOS Windows

```
python3 -m pip install --pre SomeProject
```

Installing “Extras”

Extras are optional “variants” of a package, which may include additional dependencies, and thereby enable additional functionality from the package. If you wish to install an extra for a package which you know publishes one, you can include it in the pip installation command:

Unix/macOS Windows

```
python3 -m pip install 'SomePackage[PDF]'  
python3 -m pip install 'SomePackage[PDF]==3.0'  
python3 -m pip install -e '.[PDF]' # editable project in current directory
```

- [1] “Secure” in this context means using a modern browser or a tool like **curl** that verifies SSL certificates when downloading from https URLs.
- [2] Depending on your platform, this may require root or Administrator access. [pip](#) is currently considering changing this by [making user installs the default behavior](#).
- [3] Beginning with Python 3.4, `venv` (a stdlib alternative to [virtualenv](#)) will create virtualenv environments with `pip` pre-installed, thereby making it an equal alternative to [virtualenv](#).
- [4] The compatible release specifier was accepted in [PEP 440](#) and support was released in [Setuptools](#) v8.0 and [pip](#) v6.0

↑ Наверх